75

Distributed Computation and Networks
A Performance Perspective

Assignment 4:
Performance Evaluation of Cloud Servers

Alessandro Versini *
(av2013)

March 28, 2017

*Declaration: We confirm that this submission is our own work. In it, we give references and citations whenever we refer to or use the published, or unpublished, work of others. We are aware that this course is bound by penalties as set out in the College examination offenses policy.

Contents

1 Introduction **3**

2 System Analysis **4**
2.1 Joint Probability Distribution of Queue Length 4
2.2 Average Waiting Time 4

3 System Optimisation **5**
3.1 Closed Form Optimisation 5
3.2 Average Number of Jobs in Each Server 9

4 Consider a Modified System **9**
4.1 Average Packet Delay in a Single Queue System 10
4.2 Other Properties of Single Queue System 10

1 Introduction

In the past few years there has been a tendency to move the computation out of personal devices towards very large distributed computers. This does not only have the advantage of making high performance computation accessible to very small mobile devices but, with the IT energy problem becoming increasingly serious, also provides a solution that is way more energy efficient.

Clearly these advantage comes at the cost of the need for new types of infrastructures that must ensure the correctness and completeness of the computation while at the same time making the latency as short as possible. In this Assignment we are asked to study the performance of a simple Cloud Service Provider (CSP) in terms of the average waiting time of a new job reaching the cloud and to derive ways of determining parameters that achieve optimal job scheduling.

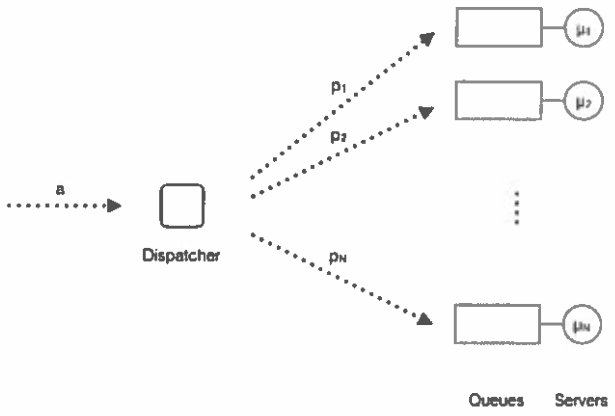


Figure 1: Diagram of a simple Cloud Service Provider.

We will start by showing the expression for the probability distribution of the queue lengths in the steady state $P(n_1, n_2, \ldots, n_N)$. From it and using Little's theorem we will derive an expression for the average waiting time of a packet arriving at the system in function of p_i[1]. We will then consider an optimisation framework to find the value of p_i that yields the minimum average waiting time. Finally we will consider a slight modification of the network and analyse its strengths and weaknesses.

[1] Probability of a job to be dispatched to server i

2 System Analysis

2.1 Joint Probability Distribution of Queue Length

To prove an expression for the joint probability distribution of the queue length in the system, we note that there is no interaction between individual servers once a job has been dispatched. It follows that the joint probability distribution can be written as a product of the marginals, as shown:

$$P(n_1, n_2, \ldots, n_N) = \prod_{i=1}^{N} P(n_i) \tag{1}$$

where the probability distribution $P(n_i)$ of the queue length at node i will be determined by the Poisson arrival rate λ_i and the departure rate μ_i as in the single router case studied in a previous Assignment.

More specifically, the probability distribution of a single M/M/1 queue has been shown to equal:

$$P(n_i) = \left(1 - \frac{\lambda_i}{\mu_i}\right) \cdot \left(\frac{\lambda_i}{\mu_i}\right)^{n_i} \tag{2}$$

Furthermore, by properties of Poisson processes the following holds: given a Poisson process A with rate a, the process Λ_i obtained by splitting A according to $\Lambda_i = p_i \cdot A$ is also Poisson with rate equal to:

$$\lambda_i = p_i \cdot a \tag{3}$$

Finally, combining (2) and (3) yields the desired expression for $P(n_i)$:

$$P(n_i) = \left(1 - \frac{a \cdot p_i}{\mu_i}\right) \cdot \left(\frac{a \cdot p_i}{\mu_i}\right)^{n_i} \tag{4}$$

2.2 Average Waiting Time

The average waiting time in the system can be calculated as a weighted sum of the waiting time in each server weighted by the probability p_i of dispatching a job from the job dispatcher to the server i. This is shown below:

$$W = \sum_{i=1}^{N} p_i \cdot W_i \tag{5}$$

where W indicates the global average waiting time and W_i the average waiting time in queue i.

It follows that in order to get an expression for W we first need to obtain one for W_i. We can proceed as follows. First we obtain an expression for the average queue length $E[n_i]$ in queue i. Then we use Little's low to get an expression for the average waiting time W_i.

Again from Assignment 2, the average queue length can be found as:

$$E[N] = \sum_{n=0}^{\infty} n \cdot P_n \tag{6}$$

which substituting the expression for $P(n_i)$ obtained in (4) and solving the infinite sum yields:

$$E[n_i] = \sum_{n=0}^{\infty} n_i \cdot P(n_i) = \frac{\frac{a \cdot p_i}{\mu_i}}{1 - \frac{a \cdot p_i}{\mu_i}} \tag{7}$$

Finally, by Little's law:

$$W_i = \frac{E[n_i]}{\lambda_i} = \frac{E[n_i]}{a \cdot p_i} = \frac{\frac{1}{\mu_i}}{1 - \frac{a \cdot p_i}{\mu_i}} \tag{8}$$

From (8) and (5) an expression for the system average waiting time follows:

$$W = \sum_{i=1}^{N} p_i \cdot \frac{\frac{1}{\mu_i}}{1 - \frac{a \cdot p_i}{\mu_i}} = \sum_{i=1}^{N} \frac{p_i}{\mu_i - a \cdot p_i} \tag{9}$$

3 System Optimisation

Now that an expression for the average waiting time has been obtained we wish to find the value of p_i (virtually the only parameter we are in control of) such that (9) is minimised.

3.1 Closed Form Optimisation

To do so we formulate the minimisation of W as an optimisation problem with (9) as the objective function:

$$\begin{aligned} \text{minimise} \quad & \sum_{i=1}^{N} \frac{p_i}{\mu_i - a \cdot p_i} \\ \text{s.t.} \quad & p^T \underline{1} = 1 \\ & p_i \geq 0 \ \forall i \end{aligned} \tag{10}$$

The Lagrangian of the problem is:

$$L(p, \lambda, \rho_1, \rho_2, \ldots, \rho_N) = \sum_{i=1}^{N} \frac{p_i}{\mu_i - a \cdot p_i} + \lambda(p^T \underline{1} - 1) + \sum_{i=1}^{N} \rho_i(-p_i) \tag{11}$$

To find the optimal solution we require $\nabla L = 0$ which can be calculated as (KKT optimality conditions):

$$\begin{cases} \frac{\mu_i}{(\mu_i - a \cdot p_i)^2} - (\rho_i - \lambda) = 0 & \forall i \\ p^T \underset{\rightarrow}{1} = 1 & \\ p_i \geq 0 & \forall i \\ \rho_i \geq 0 & \forall i \end{cases} \tag{12}$$

Clearly (12) is a non linear system of equation which is quite hard to solve. Therefore we propose two ways of tackling this problem.

3.1.1 Inactive Inequality Constraints

Assuming that the solution has all inequality constraints inactive ($\rho_i = 0 \ \forall i$) we can simplify $\nabla L = 0$ as follows:

$$\begin{cases} \frac{\mu_i}{(\mu_i - a \cdot p_i)^2} + \lambda = 0 \\ p^T \underset{\rightarrow}{1} = 1 \end{cases} \tag{13}$$

From which we can get an expression for p_i in a closed form:

$$p_i = \frac{\mu_i - \frac{\sqrt{\mu_i}}{\sum_j \sqrt{\mu_j}} (\sum_j \mu_j - a)}{a} \tag{14}$$

3.1.2 Ratios

The second solution is to rewrite the minimisation as an unconstrained optimisation problem but assuming that p_1 for the slowest server is given and to determine expressions for all p_i in function of p_1 [1]. In particular if this is done the optimisation framework becomes:

$$\text{minimise} \ \frac{p_1}{\mu_1 - a \cdot p_1} + \sum_{i=2}^{N} \frac{p_i}{\mu_i - a \cdot p_i} \tag{15}$$

Differentiating and setting to zero yields:

$$\frac{\mu_i}{(\mu_i - a \cdot p_i)^2} = \frac{\mu_1}{(\mu_1 - a \cdot p_1)^2} \tag{16}$$

which further simplifying becomes:

$$p_i = \frac{\mu_i - \sqrt{\frac{\mu_i}{\mu_1}} (\mu_1 - a \cdot p_1)}{a} \tag{17}$$

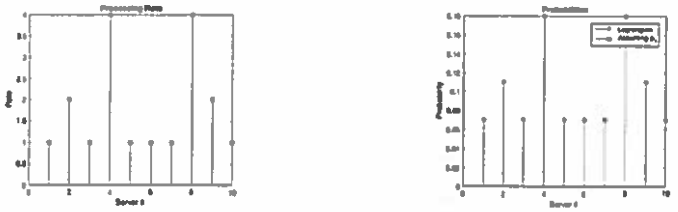
By noting that probabilities have to sum to 1, an expression for p_1 immediately follows:

$$p_1 = \frac{a + \sum_{i=1}^{N} \sqrt{\mu_i \mu_1} - \sum_{i=1}^{N} \mu_i}{a \sum_{i=1}^{N} \frac{\sqrt{\mu_i}}{\sqrt{\mu_1}}} \tag{18}$$

3.1.3 Comparison & Heuristic Packet Scheduling

We now show experimentally that the two solutions yield the same result. This is shown in Figure 2 for the following numerical example:

```
a = 50
u = [1 2 1 4 1 1 1 4 2 1]
```



(a) Processing Rate (b) Probabilities

Figure 2: Illustration of (a) processing rate of 10 servers and (b) resulting system probabilities.

Unfortunately, both solution above presented have the same limitation: in particular in both cases, inactive inequality constraints are assumed, which, if the response time of the servers is particularly uneven, can lead to negative probabilities as the result of the optimisation.

In practice it can be shown experimentally that a feasible way of choosing p is to assign a value to p_1 according to the equation:

$$p_1 = max\left(\frac{a + \sum_{i=1}^{N}\sqrt{\mu_i\mu_1} - \sum_{i=1}^{N}\mu_i}{a\sum_{i=1}^{N}\frac{\sqrt{\mu_i}}{\sqrt{\mu_1}}}, 0\right) \tag{19}$$

Since being the slowest server implies $p_1 \leq p_i \ \forall i$, this ensures that no negative probabilities will result out of the optimisation problem.

As a final remark, if the value of p_1 is found according to (19), the vector of probabilities p must be normalised as:

$$\hat{p}_i = \frac{p_i}{\sum_j p_j} \tag{20}$$

Figure 3 shows the result of using (19). The experimental setting is:

```
a = 50
u = [1 2 1 4 1 100 1 4 2 1]
```



(a) Processing Rate (b) Probabilities

Figure 3: Illustration of (a) processing rate of 10 servers and (b) resulting system probabilities.

Note that, as expected, while the standard Lagrangian solution finds an infeasible solution that contains negative probabilities, the heuristic correction proposed yields a solution that can be practically implemented.

As a closing remark, we would like to stress a very interesting result, that is that the vector of optimal probabilities $\boldsymbol{p}^*$ is not a function exclusively of the service rates but also of the input rate. What makes this result even more fascinating though is its relation to the possible existence of zero probabilities in the optimal set $\boldsymbol{p}^*$.

To understand this better, let us assume to have two servers: one slow an one fast. If the input rate is small (low utilisation) it would be always more desirable to send fresh packets to the fastest server and therefore the probability of dispatching packets to the slow server would be null. However as the rate starts increasing and packets start queuing in the faster server, the time spent by packets joining the faster queue becomes comparable to the average service time of the slow server, the probabilities of dispatching packets to the slow server has to become non-zero.

In other words, is it more desirable to join a fast but long queue or a slow but short queue? The answer in short is that there are two costs to consider: the first one is the cost of being processed (which is represented by the average service time); the second one is the cost of waiting to be served (which is represented by the queue length and not only by the service time). In a slow server, usually few jobs are dispatched and therefore the first cost prevails whereas in a faster server where often many jobs are dispatched the second cost tends to be more significant.

We conclude this general comment on the performance of the system by noting that as the input rate a increases (although in practice it should never become larger than $\sum_i \mu_i$), the value of $\boldsymbol{p}^*$ tends to be less dependent on a and more

on the vector of service rates μ:

$$\lim_{a\to\infty} p_i^* = \frac{\sqrt{\mu_i}}{\sum_j \sqrt{\mu_j}} \tag{21}$$

Note furthermore that p_i^* must be non-zero (assuming non-zero service times). This confirms that for higher input rates the chances of getting a zero in the optimal probability vector decreases.

3.2 Average Number of Jobs in Each Server

Once the optimal probabilities have been calculated, getting an expression for the average number of jobs in each of the servers i is trivial. In fact we have already determined it in (6) and to obtain the average number of jobs in the queue i under optimal p it is enough to substitute p_i with the output of the minimisation p_i^*.

In particular, from (17) let p_i^* be:

$$p_i^* = \frac{\mu_i - \sqrt{\frac{\mu_i}{\mu_1}}(\mu_1 - a \cdot p_1)}{a} \tag{22}$$

It follows that the expected number of jobs in each server i is:

$$E[n_i] = \frac{\mu_i - \sqrt{\frac{\mu_i}{\mu_1}}(\mu_1 - ap_1)}{\sqrt{\frac{\mu_i}{\mu_1}}(\mu_1 - ap_1)} \tag{23}$$

which further simplifying reduces to:

$$E[n_i] = \frac{\mu_i}{\sqrt{\frac{\mu_i}{\mu_1}}(\mu_1 - ap_1)} - 1 \tag{24}$$

$2 \leq i \leq N$

4 Consider a Modified System

Once the analysis of the Cloud Service Provider is complete, we are asked to discuss the impact of a slight modification of the original system. In particular, let us consider a system which has a single queue and where the job dispatcher sends packets only to those servers whose previous task has been completed. We consider several implications of this new architecture (hereby referred as (B)) and we compare them with the original system (A).

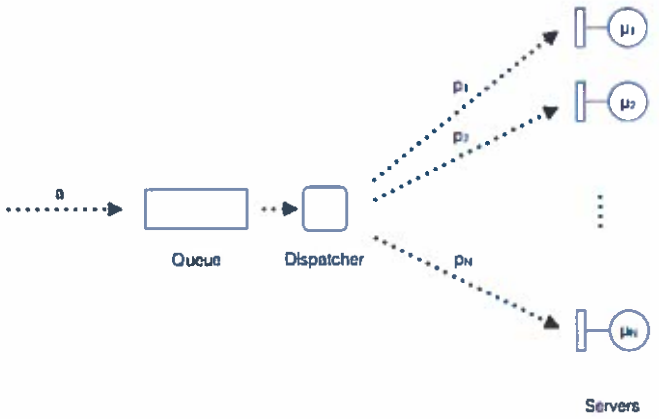


Figure 4: Diagram of modified Cloud Service Provider.

4.1 Average Packet Delay in a Single Queue System

The metric that we used to assess the performance of system (A) in previous sections was the average packet delay. In this paragraph we try to use some logical arguments to compare, in terms of the same metric, the performance of the two systems.

The first thing we notice is that system (B) tries to optimise the utilisation of all servers, dispatching fresh packets to any server that is in idle state. This might seem, at first, a strength but in actual fact it is a limitation. In fact it was shown in previous sections that maximising the overall server utilisation does not imply a maximisation of the average waiting time (e.g. some servers could receive packets with zero probability at the optimal point).

It follows that the system as it is does not work optimally and it can be shown that this holds especially for low input rates. For higher input rates, the optimal probability in system (A), as it was shown before, depends less on the overall input rate a and more on the vector μ; therefore, in this case we might expect system (B) (whose dispatching intrinsically depends on μ) to approach the performance of system (A). In either case a detailed quantitative comparison of the two system is extremely involved and depends on a significant number of variables.

4.2 Other Properties of Single Queue System

Although this Assignment was particularly concerned with the latency of the system, the average packet delay is not the only metric that cloud service

providers would consider when designing such infrastructures. Below we present a few examples of typical constraints faced during CSP design.

4.2.1 Peak to Mean Energy Consumption

As it often happen in engineering, IT infrastructures are designed to deal with peak rather than with mean utilisation. This implies that, out of peak hours, a large number of servers could be significantly underutilised in which case it would be more efficient to power some of the processing units down and schedule new jobs on fewer machines. Clearly, architecture (B) with a single large queue does not offer such a flexibility.

4.2.2 Technical Feasibility

Another quite basic but important aspect to consider is that large queues are, quite intuitively, difficult to manage: they need large amounts of storage space and they are generally slower to access than smaller queues.

4.2.3 Reliability

Finally, one last point in favour of the original system (A) is the reliability of the system. In fact having multiple queues lowers the probability of a fault taking all servers down as it can happen in architecture (B) if a fault occurs in the central queue.

References

[1] Ricardo Lent Erol Gelenbe. Optimising server energy consumption and response time. *Theoretical and Applied Informatics*, 24(4), 2012.